Web Accessibility

(A11y)

Designing for ALL users — visual, motor, cognitive, hearing

K0126002 · Gachon University · 2026 Fall · Dr. Toshpulatov

What is Accessibility (A11y)?

Visual

Blindness · Low vision · Color blindness

~2.2 billion people globally

Motor

Tremors · Limited arm mobility · No mouse

Keyboard-only navigation essential

Cognitive

Dyslexia · ADHD · Memory impairments

Clear language, consistent layouts

Hearing

Deafness · Hard of hearing

Captions, transcripts, visual cues

1 in 6 people worldwide has a disability. Accessible sites also rank better in SEO, load faster, and convert more users.

WCAG — Web Content Accessibility Guidelines

P

Perceivable

Users can see, hear, or feel all content

- Alt text on images
- Captions on video
- Sufficient color contrast

O

Operable

Users can navigate without a mouse

- Full keyboard access
- No keyboard traps
- Skip navigation links

U

Understandable

Content is clear and predictable

- Consistent navigation
- Plain language
- Helpful error messages

R

Robust

Works with current and future AT

- Valid semantic HTML
- ARIA used correctly
- Future-proof markup

Target for this course: WCAG 2.1 Level AA (Level A = minimum, AA = industry standard, AAA = maximum)

💡 QUICK QUIZ

WCAG 2.1 Level AA requires a minimum contrast ratio of ___ for normal body text.

A 2:1

B 3:1

C 4.5:1

D 7:1

🗨️ *Discuss with your neighbor — then we'll reveal the answer!*

✅ ANSWER

WCAG 2.1 Level AA requires a minimum contrast ratio of ___ for normal body text.

A 2:1

B 3:1

C 4.5:1

D 7:1

WCAG AA requires 4.5:1 for text under 18pt. Large text (18pt+ or 14pt+ bold) only needs 3:1.
Level AAA raises it to 7:1 for all text. Tool: WebAIM Contrast Checker or Chrome DevTools.

Semantic HTML vs Div Soup

Meaning beats markup

❌ Div Soup — no meaning

```
<div class="header">
  <div class="nav">
    <div>Home</div>
    <div>About</div>
  </div>
</div>
<div class="main">
  <div class="article">
    <div class="h1">Title</div>
    <div>Content here</div>
  </div>
</div>
<div class="footer">© 2026</div>
```

Screen reader announces:

"group"
"group"
"Home, link"
"group"
"group"

✅ Semantic HTML — clear meaning

```
<header>
  <nav>
    <a href="/">Home</a>
    <a href="/about">About</a>
  </nav>
</header>
<main>
  <article>
    <h1>Article Title</h1>
    <p>Content here</p>
  </article>
</main>
<footer>Copyright 2026</footer>
```

Screen reader announces:

"banner landmark"
"navigation landmark"
"Home, link"
"main landmark"
"Article Title, heading level 1"

💡 QUICK QUIZ

Which element is the most accessible way to mark up a site's navigation links?

A

B <ul id='navigation'>

C <nav>

D <menu>

🗨️ *Discuss with your neighbor — then we'll reveal the answer!*

✅ ANSWER

Which element is the most accessible way to mark up a site's navigation links?

A

B <ul id='navigation'>

C <nav>

D <menu>

<nav> is a semantic landmark element. Screen readers announce it as "navigation" and let users jump directly to it with a single keypress (N in NVDA). carries no semantic meaning whatsoever.

<button> vs <div onclick> — Why Native Elements Matter

❌ <div onclick>

```
<div onclick="doSomething()"
     style="cursor:pointer">
  Click me
</div>
```

VS

✅ <button>

```
<button
  type="button"
  onclick="doSomething()">
  Click me
</button>
```

❌ Problems

- Cannot be focused with Tab key
- Enter / Space keys don't fire click
- No accessible role assigned
- Screen reader: "Click me, group"

✅ Benefits

- Focusable with Tab key (built-in)
- Enter AND Space both fire click
- Role "button" auto-announced
- Screen reader: "Click me, button"

Rule: Use <button> for actions, <a href> for navigation. Never use a div or span for interactive elements.

💡 QUICK QUIZ

A designer makes a clickable that opens a modal. What is MISSING for accessibility?

A A CSS class

B display: flex styling

C Keyboard focus + Enter/Space support + a role

D A data-attribute

💭 *Discuss with your neighbor — then we'll reveal the answer!*

✅ ANSWER

A designer makes a clickable that opens a modal. What is MISSING for accessibility?

A A CSS class

B display: flex styling

C Keyboard focus + Enter/Space support + a role

D A data-attribute

Divs get no keyboard focus or role by default. Use <button> instead, or add role="button" tabindex="0" and a keydown handler for Enter/Space. Without these, keyboard users and screen reader users cannot activate the control at all.

Alt Text on Images — When and What to Write

```
<!-- ✅ Informative: describe content -->
<img src="chart.png"
     alt="Bar chart: 40% rise in
          enrollment 2024-2026">

<!-- ✅ Decorative: empty alt -->
<img src="divider.svg" alt="">

<!-- ✅ Functional icon button -->
<button>
  <img src="search.svg" alt="Search">
</button>

<!-- ❌ Useless alt text -->
<img src="logo.png"
     alt="image of a logo">
```

Informative

Describe the content AND purpose. What would someone miss if they couldn't see it?

Decorative → alt=""

Empty alt tells screen readers to skip it completely. No alt at all makes them announce the filename.

Functional (buttons)

Describe the action, not the appearance. alt="Search" not alt="magnifying glass icon".

Never write...

"image of..." or "photo of..." — screen readers already announce "image". It's redundant and wastes attention.

💡 QUICK QUIZ

An image is purely decorative (a background swirl). What is the correct alt attribute?

A alt='decorative image'

B alt='' (empty string)

C No alt attribute at all

D alt='background'

💭 *Discuss with your neighbor — then we'll reveal the answer!*

✅ ANSWER

An image is purely decorative (a background swirl). What is the correct alt attribute?

A alt='decorative image'

B alt='' (empty string)

C No alt attribute at all

D alt='background'

alt='' (empty string) tells screen readers to skip this image completely. A missing alt attribute causes screen readers to announce the raw filename ("IMG_background_swirl.jpg"). alt='decorative image' is still read aloud — redundant and wrong.

ARIA Roles & Attributes — When HTML is Not Enough

```
<!-- Name a landmark region -->
<nav aria-label="Main navigation">
  ...
</nav>

<!-- Hide decorative icon -->
<span aria-hidden="true">★</span>
<span>Rated 4.8 out of 5</span>

<!-- Show open/closed state -->
<button aria-expanded="false"
        aria-controls="menu">
  Menu
</button>
<ul id="menu" hidden>...</ul>
```

🥇 **FIRST RULE: Use semantic HTML before ARIA. ARIA should add, never replace.**

`aria-label`

Gives a visible or hidden name to an element when no text label is present

`aria-describedby`

Links an element to a longer description (e.g., error message, hint text)

`aria-hidden="true"`

Hides decorative or redundant elements from screen readers only — still visible on screen

`aria-expanded`

Announces whether a collapsible widget (menu, accordion) is open or closed

`role="..."`

Overrides or assigns a semantic role when native elements aren't available

💡 QUICK QUIZ

What does aria-hidden='true' do to an element?

A Makes it invisible on screen

B Removes it from screen reader announcements

C Disables all click events

D Makes it transparent with CSS

💭 *Discuss with your neighbor — then we'll reveal the answer!*

✅ ANSWER

What does aria-hidden='true' do to an element?

A Makes it invisible on screen

B Removes it from screen reader announcements

C Disables all click events

D Makes it transparent with CSS

aria-hidden='true' removes the element from the accessibility tree (what screen readers see) WITHOUT affecting its visual display. Perfect for decorative icons (★ 🤍 ✨) or repeated UI elements that would add noise for screen reader users.

Keyboard Navigation — It Must ALL Work

Tab	Move to next focusable element
Shift + Tab	Move to previous focusable element
Enter	Activate links and buttons
Space	Check/uncheck boxes; activate buttons
↑ ↓ ← →	Navigate menus, radio groups, sliders
Escape	Close modals and dropdown menus

Quick test: unplug your mouse (or put it away). Open your site and navigate using only the keyboard. Can you reach every link and button?

tabindex="0" — Making Custom Elements Focusable

```
<div
  role="button"
  tabindex="0"
  aria-label="View project details"
  onclick="openProject()"
  onkeydown="handleKey(event)">
  <img src="thumb.jpg" alt="">
  <p>My Final Project</p>
</div>

function handleKey(e) {
  if (e.key === "Enter" ||
      e.key === " ") {
    openProject();
    e.preventDefault(); // stop page scroll
  }
}
```

tabindex="0"

Adds to tab order at natural DOM position. Use for custom interactive elements (cards, custom buttons).

tabindex="-1"

Focusable with JavaScript (element.focus()) but NOT reached by Tab. Use for modal focus management.

tabindex="1+"

Positive values DISRUPT the natural tab order. Avoid! Users Tab in unexpected order.

role=

Always pair tabindex="0" with the correct ARIA role. Without role, "0" adds focus but no meaning.

onkeydown

Manually handle Enter and Space. Remember e.preventDefault() for Space — it otherwise scrolls the page.

Color Contrast — WCAG AA Requires 4.5:1

Sample text Aa 01

Sample body text at 14pt

FAIL

2.0:1

Light gray on white

Sample text Aa 01

Sample body text at 14pt

PASS AA

4.5:1

#767676 on white (bare minimum)

Sample text Aa 01

Sample body text at 14pt

PASS AAA

15.3:1

White on our slide navy

Sample text Aa 01

Sample body text at 14pt

PASS AAA

9.4:1

Gold on navy (this deck accent)

Tool: WebAIM Contrast Checker — webaim.org/resources/contrastchecker/ or Chrome DevTools → CSS Overview → Colors

Skip Navigation Links — Jump to Main Content

```
<!-- First element in <body> -->
<a href="#main-content"
   class="skip-link">
  Skip to main content
</a>

<nav>...(many links)...</nav>

<main id="main-content">
  <!-- page content -->
</main>

/* Hidden until focused */
.skip-link { position: absolute; left: -9999px; }
.skip-link:focus {
  left: 0.5rem; top: 0.5rem;
  z-index: 9999;
  background: #1a2744;
  color: white;
  padding: 0.5rem 1rem;
}
```

1 — Why needed

Keyboard users must Tab through every nav link before reaching content. A 47-link nav = 47 Tab presses.

2 — How it works

The link is visually hidden (far left: -9999px) until a user presses Tab. Then it appears at top-left.

3 — Implementation

Place it as the FIRST element in <body>. The href points to id="main-content" on <main>.

4 — Test it

Press Tab once on any page. If a "Skip to content" link appears — the site has it. If not — it's missing.

Lighthouse Accessibility Audit

1. Open Chrome DevTools (F12)
2. Click the "Lighthouse" tab
3. Check "Accessibility" only
4. Click "Analyze page load"
5. Read the score (0–100)
6. Expand each failed audit
7. Click "Learn more" links
8. Fix and re-run to verify

Common failures: missing alt text · insufficient contrast · unlabeled form inputs · missing <html lang="en">

95

target score

Summary & Homework

Semantic HTML

<header> <nav> <main> <article> — structure with meaning

Native Elements

<button> for actions, <a href> for links — always over div

Alt Text

Describe purpose; alt="" for decorative; never "image of..."

ARIA Attributes

aria-label, aria-hidden, aria-expanded — use sparingly

Keyboard Navigation

Tab, Enter, Space, Arrows, Escape must ALL work

Color Contrast

WCAG AA: 4.5:1 minimum for body text

HOMEWORK

Due: Thursday before class · Post to Discord

1. Run Lighthouse on your personal, team project site (Accessibility audit only)
2. Screenshot the initial Accessibility score(make sure the scores are 100)
3. Fix at least 3 failing audits from the report
4. Re-run Lighthouse and screenshot the improved score
5. Post both screenshots + list of 3 fixes to the weekly task page

Add a skip link to their course project. It takes about 10 lines of HTML + CSS and satisfies a WCAG Level A requirement.

StudyGuide.html

VideoLecture.html

Accessibility_v3.pdf

Quiz